

Laboratorium 6

Dane sonarowe BSS i klasyfikacja dna morskiego

Igor Józefowicz, indeks 193257

1. Dane i cel

Raport dotyczy ćwiczenia z przedmiotu *Uczenie maszynowe w badaniach Ziemi*. Celem było obliczenie parametrów opisujących sondowania BSS z sonaru wielowiązkowego, pokazanie ich rozkładów oraz sprawdzenie skuteczności prostych klasyfikatorów dla czterech typów dna.

Wykorzystano plik `dane_bss.mat`, który zawiera tablicę `vbss` o rozmiarze $4 \times 600 \times 160$: cztery typy dna, 600 sondowań dla każdego typu i 160 wiązek sonaru w jednym sondowaniu. Do obliczeń użyto pierwszych 500 sondowań z każdej klasy, zgodnie z dostarczonymi skryptami.

Typy dna: 1 – muł, 2 – mieszanina mułu, piasku i odpadów antropogenicznych, 3 – piasek drobnoziarnisty, 4 – piasek gruboziarnisty.

Najlepszy wynik klasyfikacji w wykonanych próbach uzyskał model `knn_5` dla wszystkich parametrów i 60% danych uczących: `accuracy=96.00%`.

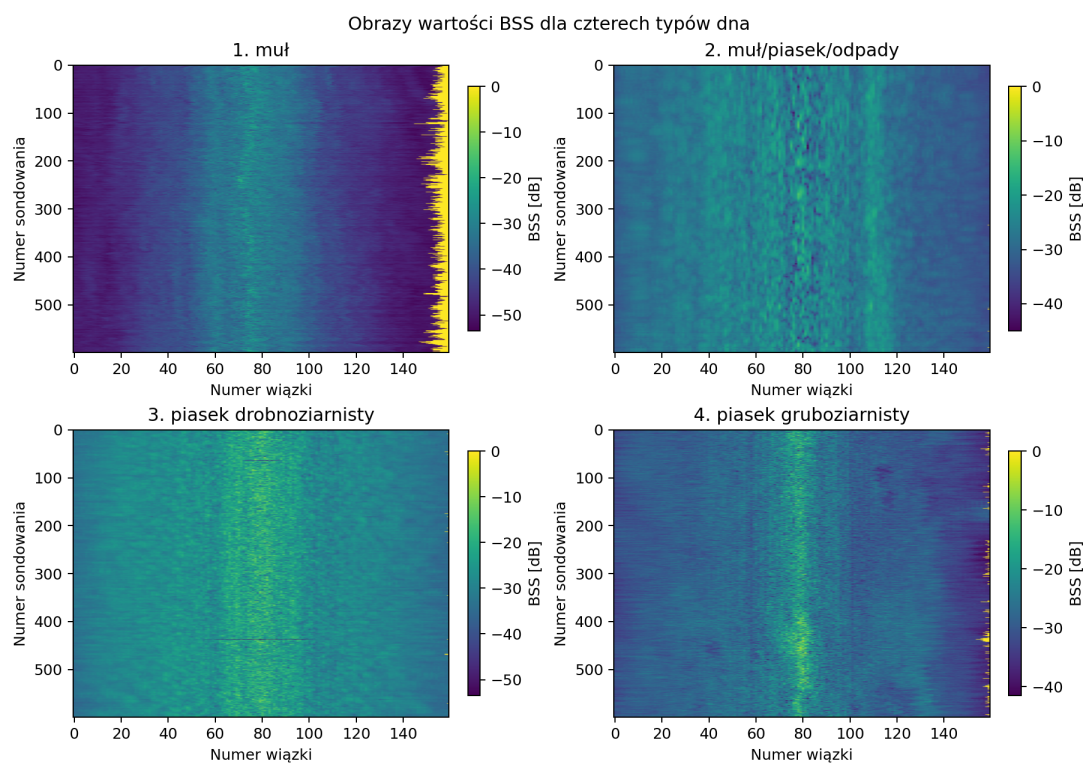
2. Obliczone parametry

Obliczono cztery parametry wymagane w instrukcji oraz trzy lekkie rozszerzenia. W tabeli zastosowano skrócone nagłówki: GLCM E oznacza entropię, GLCM LH spójność lokalną, L-R różnicę sektorów lewego i prawego.

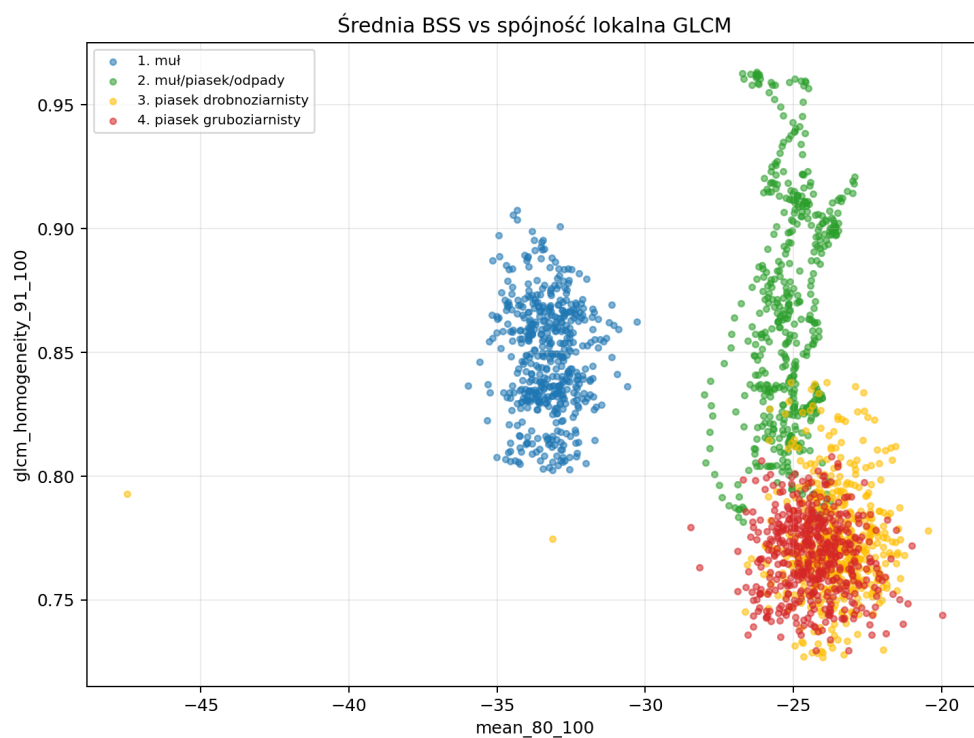
typ dna	mean	drop	GLCM E	GLCM LH	std	L-R	edge-center
1. muł	-33.29	7.17	0.58	0.85	2.71	3.59	-15.28
2. muł/piasek/odpady	-25.16	0.89	0.52	0.86	3.38	0.14	-3.92
3. piasek drobnoziarnisty	-23.68	6.63	0.77	0.77	4.26	1.99	-6.76
4. piasek gruboziarnisty	-24.34	9.64	0.70	0.77	4.58	3.25	-5.70

3. Wizualizacja parametrów

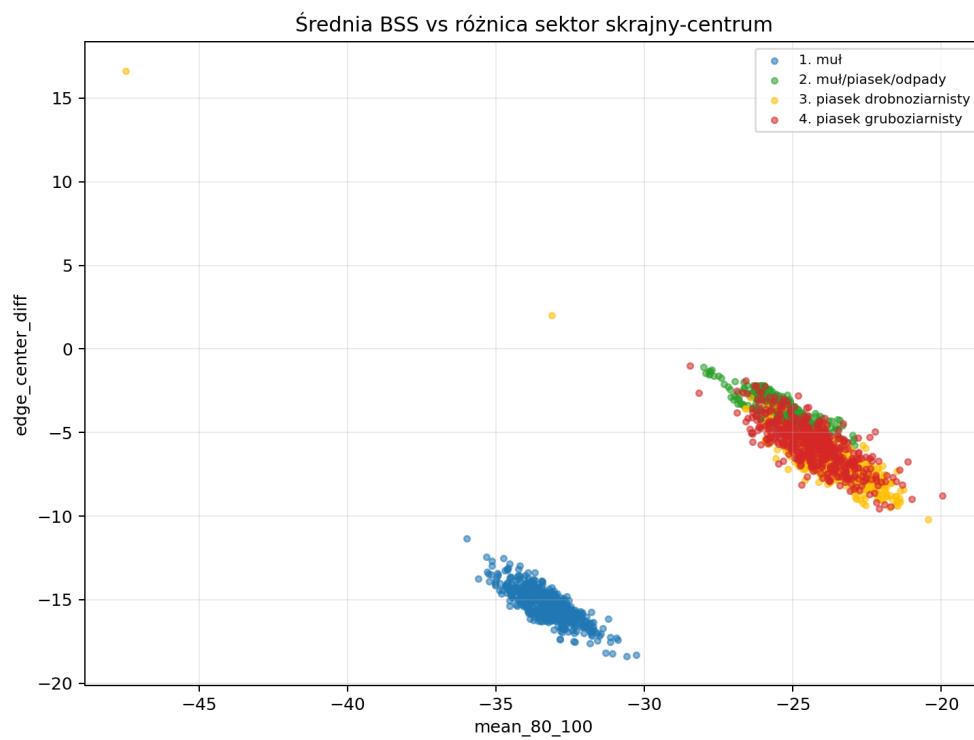
Wykresy pokazują, że klasa 1 jest najlepiej oddzielona od pozostałych przez poziom BSS. Klasy 2, 3 i 4 częściowo nakładają się w przestrzeni parametrów, dlatego sama średnia BSS nie wystarcza do stabilnej klasyfikacji. Dodatkowe parametry opisują kształt zmian sygnału wraz z kątem sondowania i lokalną teksturę obrazu sonarowego.



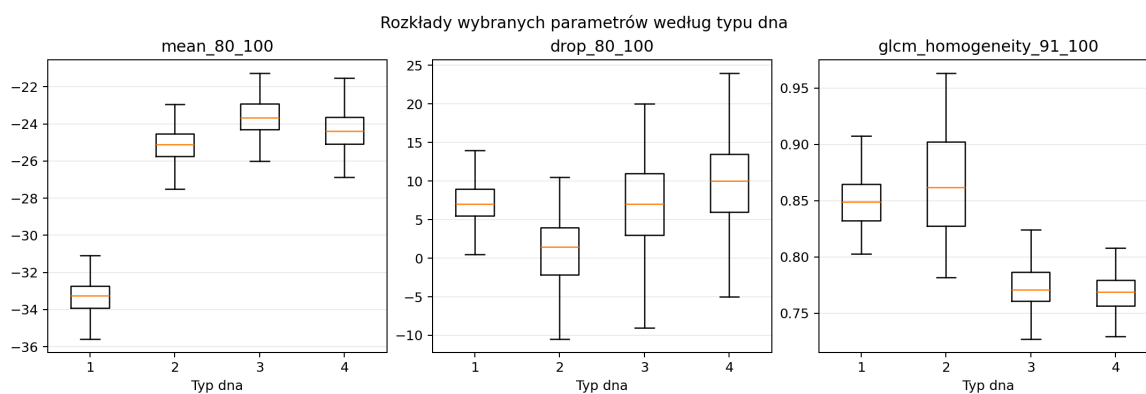
Rysunek 1: Obrazy BSS dla czterech typów dna.



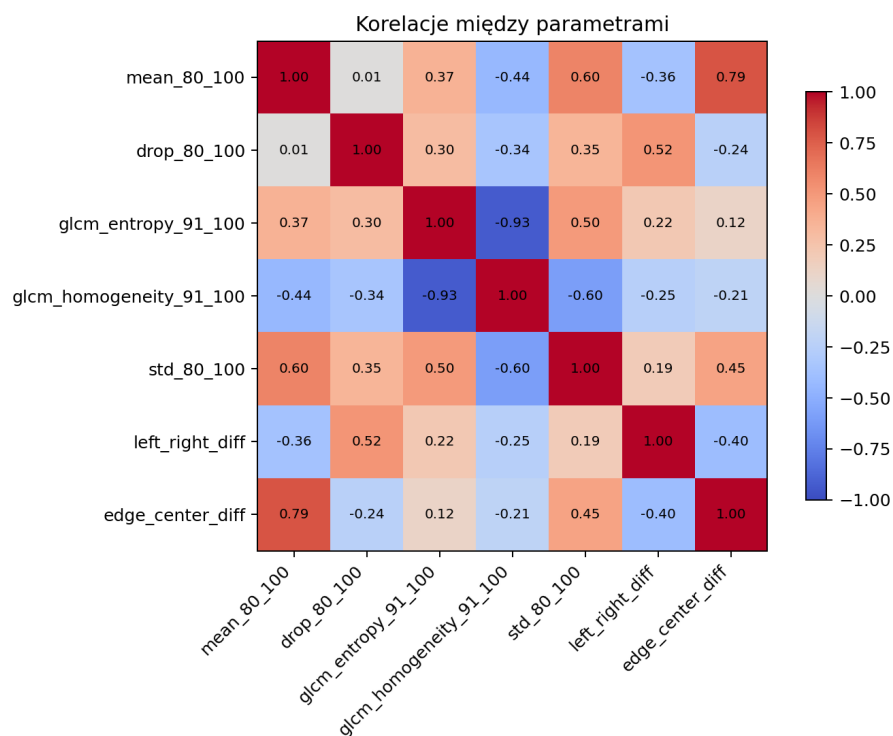
Rysunek 2: Rozkład 2D: średnia BSS i spójność lokalna GLCM.



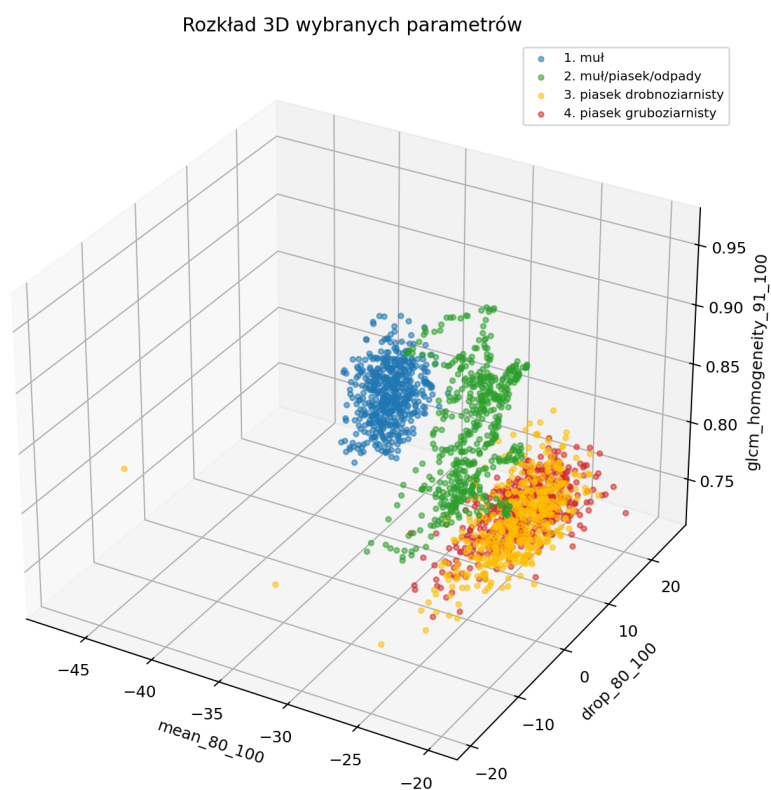
Rysunek 3: Rozkład 2D: średnia BSS i różnica sektorów.



Rysunek 4: Rozkłady wybranych parametrów według typu dna.



Rysunek 5: Korelacje między obliczonymi parametrami.



Rysunek 6: Dodatkowy rozkład 3D dla trzech parametrów.

4. Klasyfikacja

Klasyfikację minimalnoodległościową odtworzono przez **NearestCentroid**. Dane normalizowano metodą min-max, a podział uczący/testowy wykonano per klasa: pierwsze 20%, 40% albo 60% obiektów jako zbiór uczący, reszta jako testowy.

Pary parametrów

model	parametry	trening [%]	accuracy [%]
min. odl.	mean + GLCM LH	40	77.58
min. odl.	mean + GLCM LH	60	75.25
min. odl.	mean + drop	60	70.88
min. odl.	mean + edge-center	20	69.62
min. odl.	mean + drop	40	68.92
min. odl.	mean + drop	20	67.44
min. odl.	mean + edge-center	40	67.17
min. odl.	mean + edge-center	60	65.12
min. odl.	mean + GLCM LH	20	63.19

Wszystkie parametry

model	parametry	trening [%]	accuracy [%]
kNN	wszystkie	60	96
kNN	wszystkie	40	92.50
min. odl.	wszystkie	60	89.75
kNN	wszystkie	20	86.69
min. odl.	wszystkie	40	84.75
min. odl.	wszystkie	20	76.44

Najlepsza para dla klasyfikatora minimalnoodległościowego przy 20% treningu to **mean+edge_center**, accuracy=69.62%.

Tabela 1: Najlepsza para parametrów, 20% treningu

Liczba obiektów					Udział procentowy [%]				
	kl. 1	kl. 2	kl. 3	kl. 4		kl. 1	kl. 2	kl. 3	kl. 4
rzecz. 1	400	0	0	0	rzecz. 1	100.00	0.00	0.00	0.00
rzecz. 2	0	245	21	134	rzecz. 2	0.00	61.25	5.25	33.50
rzecz. 3	0	18	328	54	rzecz. 3	0.00	4.50	82.00	13.50
rzecz. 4	0	68	191	141	rzecz. 4	0.00	17.00	47.75	35.25

Tabela 2: NearestCentroid, wszystkie parametry, 20% treningu

Liczba obiektów					Udział procentowy [%]				
	kl. 1	kl. 2	kl. 3	kl. 4		kl. 1	kl. 2	kl. 3	kl. 4
rzecz. 1	400	0	0	0	rzecz. 1	100.00	0.00	0.00	0.00
rzecz. 2	0	271	40	89	rzecz. 2	0.00	67.75	10.00	22.25
rzecz. 3	0	0	333	67	rzecz. 3	0.00	0.00	83.25	16.75
rzecz. 4	0	0	181	219	rzecz. 4	0.00	0.00	45.25	54.75

Tabela 3: kNN, wszystkie parametry, 20% treningu

Liczba obiektów					Udział procentowy [%]				
	kl. 1	kl. 2	kl. 3	kl. 4		kl. 1	kl. 2	kl. 3	kl. 4
rzecz. 1	400	0	0	0	rzecz. 1	100.00	0.00	0.00	0.00
rzecz. 2	0	323	36	41	rzecz. 2	0.00	80.75	9.00	10.25
rzecz. 3	0	1	344	55	rzecz. 3	0.00	0.25	86.00	13.75
rzecz. 4	0	0	80	320	rzecz. 4	0.00	0.00	20.00	80.00

5. Dodany kod własny

Pliki startowe do laboratorium zawierały skrypty MATLAB/Octave, m.in. `vbss.m`, `obl_srednia.m`, `obl_spadek_dB.m`, `obl_GLCM_entr_spoj.m`, `rozkl2D.m` oraz `kl_min_dist_2D.m`. Nowy kod dodany w ramach rozwiązania znajduje się w pliku `lab6_solution.py`. Poniżej pokazano tylko najważniejsze fragmenty: obliczenie parametrów, obliczenie parametrów GLCM oraz wspólną funkcję klasyfikacji.

Listing 1: Fragment własnego kodu: obliczanie parametrów BSS.

```
def compute_features(vbss):
    values = replace_zero_with_min(vbss)
    arrays = {
        "mean_80_100": values[:, :N_OBJECTS, beam_slice(80, 100)].mean(axis=2),
        "drop_80_100": values[:, :N_OBJECTS, 79] - values[:, :N_OBJECTS, 99],
        "std_80_100": values[:, :N_OBJECTS, beam_slice(80, 100)].std(axis=2),
        "left_right_diff": values[:, :N_OBJECTS, beam_slice(61, 80)].mean(axis=2)
            - values[:, :N_OBJECTS, beam_slice(91, 100)].mean(axis=2),
        "edge_center_diff": values[:, :N_OBJECTS, beam_slice(1, 20)].mean(axis=2)
            - values[:, :N_OBJECTS, beam_slice(80, 100)].mean(axis=2),
    }
```

Listing 2: Fragment własnego kodu: obliczanie entropii i spójności lokalnej GLCM.

```
for class_idx in range(4):
    for object_idx in range(N_OBJECTS):
        window = values[class_idx, object_idx : object_idx + 30, beam_slice(91, 100)]
        entropy, homogeneity = glcm_entropy_homogeneity(
            window, levels=8, distance=1, total_min=total_min, total_max=total_max
        )
        glcm_entropy[class_idx, object_idx] = entropy
        glcm_homogeneity[class_idx, object_idx] = homogeneity
```

Listing 3: Fragment własnego kodu: wspólna funkcja klasyfikacji i ewaluacji.

```
def run_classifier(df, feature_names, train_pct, classifier_name):
    x = MinMaxScaler().fit_transform(df[feature_names].to_numpy())
    y = df["class_id"].to_numpy()
    train_idx, test_idx = split_by_class(train_pct, y)
    classifier = NearestCentroid() if classifier_name == "nearest_centroid" else
    KNeighborsClassifier(n_neighbors=5)
    classifier.fit(x[train_idx], y[train_idx])
    predictions = classifier.predict(x[test_idx])
    return accuracy_score(y[test_idx], predictions) * 100, confusion_matrix(y[test_idx],
    predictions)
```

6. Wnioski

1. Klasa 1, czyli muł, jest najłatwiejsza do rozpoznania. W głównych macierzach niezgodności wszystkie lub prawie wszystkie obiekty tej klasy trafiają do poprawnej klasy.
2. Największe pomyłki występują między klasami 2, 3 i 4, co zgadza się z wykresami parametrów, gdzie rozkłady tych klas częściowo się nakładają.
3. Najlepsza para parametrów przy 20% treningu uzyskała $\text{accuracy}=69.62\%$, więc klasyfikacja 2D jest użyteczna, ale ograniczona.
4. Najlepszy wynik całego zestawienia to `knn_5` dla wszystkich parametrów, $\text{train_pct}=60\%$, $\text{accuracy}=96.00\%$.
5. Dla `NearestCentroid` na wszystkich parametrach accuracy wzrosło od 76.44% do 89.75% , więc większy zbiór uczący poprawił stabilność klasyfikacji.
6. Dodatkowe parametry własne są przydatne, bo opisują nie tylko średni poziom BSS, lecz także zmianę sygnału wraz z kątem sondowania i lokalną teksturę obrazu.